PHASE 3 — SECURITY & ACCESS CORE · CHAPTER 10

HashiCorp Vault

Secrets Management Deep Dive

A bank safe deposit box for the entire lab — secrets go in, applications authenticate, every access is logged, and nothing sits in plaintext.



Seal / Unseal



Tokens



KV & PKI



Audit Log

CHAPTER GOAL

Centralizing Secrets on sec-vault-01

Deploy HashiCorp Vault on sec-vault-01 (10.0.2.13) as the centralized secrets management system for the entire lab. Instead of secrets scattered in config files, environment variables, and code, everything goes through Vault.

- Vault is unsealed and operational
- All lab services retrieve secrets from Vault
- Database credentials, API keys, TLS certs are stored centrally
- Secrets are encrypted at rest and in transit
- All secret access is logged and auditable
- Automatic secret rotation is configured

Time estimate: 4–5 hours

Where Are Your Secrets Now?

Secrets in Code

```
DATABASE_PASSWORD = "..."
```

- ✕ Visible in Git history forever
- ✕ Accessible to anyone with code access
- ✕ Can't rotate without redeploy

Secrets in Env Vars

```
export DATABASE_PASSWORD="..."
```

- ✕ Visible in ps aux output
- ✕ No encryption, no audit log
- ✕ Hard to rotate

Secrets in Config Files

```
password: mysql-password-123
```

- ✕ Must protect file perms manually
- ✕ Backups spread secrets accidentally
- ✕ No rotation mechanism

A Bank Safe Deposit Box for Secrets

You put secrets in
(lock them away); applications request them, authenticate, and get verified —
Vault returns the secret only if authorized, logs every access, and rotates automatically.



Vault encrypts everything — at rest and in transit — so a stolen disk or intercepted request reveals nothing without a valid, authorized identity.

Vault Architecture at a Glance

Seal / Unseal

Born "sealed" (encrypted); must be unsealed to use

Initialization

Generate unseal keys + root token — done once, securely

Authentication

Prove your identity: token, OIDC, LDAP, etc.

Authorization

What secrets can this identity access?

Secrets Engine

Plugin that stores/generates secrets: KV, Database, PKI, SSH

Engine Path

Where a secrets engine is mounted: secret/, database/

Lease

How long a secret is valid before expiration (e.g. 1h TTL)

Audit Log

Record of every secret access: who, when, what, result

Getting Vault onto sec-vault-01

Install the Vault binary and prepare a dedicated system user with locked-down directories.

1 · Install Binary

```
$ wget vault_1.15.0_linux_amd64.zip

$ sudo mv vault /usr/local/bin/

$ vault version
```

2 · System User

```
$ useradd --system --shell nologin vault

$ mkdir -p /var/lib/vault /var/log/vault

$ chmod 700 on all vault directories
```

vault.hcl & TLS Certificate

vault.hcl

```
listener "tcp" {
  address = 0.0.0.0:8200
  tls_cert_file = vault.crt
  tls_key_file  = vault.key
}

storage "file" {
  path = /var/lib/vault/data
}

ui = true
max_lease_ttl = 768h
default_lease_ttl = 168h
```

Self-Signed TLS

Vault requires TLS — all connections are encrypted, even on the lab network.

```
openssl req -x509 \
  -newkey rsa:2048 \
  -keyout vault.key \
  -out vault.crt \
  -days 1825 -nodes \
  -subj "/CN=vault.devsecops.com"
```

Systemd Service & Start

Vault runs as a hardened systemd service under the dedicated vault user — never as root.



Hardening Flags

```
ProtectSystem=full
ProtectHome=yes
NoNewPrivileges=yes
AmbientCapabilities=CAP_IPC_LOCK
```



Resilience

```
Restart=on-failure
RestartSec=5
KillMode=process
LimitNOFILE=65536
```



Go Live

```
systemctl daemon-reload
systemctl enable vault
systemctl start vault
```

Generate Unseal Keys & Root Token

```
$ vault operator init \
    -key-shares=5 \
    -key-threshold=3
```

5 keys, threshold 3

Distribute 5 unseal keys to 5 trusted people; any 3 together can unseal Vault. No single person holds the whole key.

```
Unseal Key 1: xxxxxxxx...
Unseal Key 2: xxxxxxxx...   ...   Unseal Key 5: xxxxxxxx...
Initial Root Token: s.xxxxxxxx...
```



CRITICAL: Save these keys in a password manager or encrypted file — this is the ONLY copy. Anyone with 3 keys can unseal Vault.

Unseal, Authenticate & Open the Firewall

Unseal (x3)

vault operator unseal — run 3 times with 3 different keys

Authenticate

export VAULT_TOKEN=… then vault token lookup

Open Firewall

ufw allow from 10.0.2.0/24 to any port 8200

vault status after 3 unseals

```
Sealed: false   ·   Total Shares: 5   ·   Threshold: 3   ·   Progress: 0/3
```

Plugins for Different Secret Types



KV (v2)

`secret/`

Generic secrets — API keys, passwords, versioned + soft delete



Database

`database/`

Enabled and ready to generate dynamic DB credentials



PKI

`pki/`

Automatic certificate generation with a self-signed root CA

Storing & Retrieving a Secret

Put a Secret

```
vault kv put secret/database/mysql \
  username=devops \
  password="SecurePassword123!" \
  host="db.example.com" \
  port=3306
```

KV v2 keeps version history and supports soft deletes — nothing is silently lost.

Retrieve It

```
vault kv get secret/database/mysql
```

```
host      db.example.com
password  SecurePassword123!
port      3306
username  devops
```

Automatic Certificate Generation

Vault can act as a certificate authority — issuing, tracking, and eventually revoking TLS certificates for lab services.

Enable PKI

```
vault secrets enable pki
```

Set Lifetime

```
vault secrets tune
-max-lease-ttl=87600h pki
```

Generate Root CA

```
vault write
pki/root/generate/internal
common_name="Vault Root CA"
```

Policies & Application Tokens

app-policy.hcl

```
path "secret/data/app/*" {
  capabilities = ["read","list"]
}

path "secret/metadata/*" {
  capabilities = ["list"]
}

path "pki/issue/app" {
  capabilities = ["create","update"]
}
```

Generate a Token

```
vault token create
  -policy=app-policy -ttl=8760h
```

```
token            s.xxxxxxxx...
token_duration   8760h
token_renewable  true
token_policies   [app-policy]
```

OIDC Authentication & Audit Logging

OIDC via Authentik

```
vault auth enable oidc
vault write auth/oidc/config \
  oidc_discovery_url=...authentik...
  default_role="default"
```

Apps and users in other VMs authenticate with Authentik-issued OIDC tokens instead of static Vault tokens.

Audit Logging

```
vault audit enable file \
  file_path=/var/log/vault/audit.log
```

Every API call — reading secrets, authenticating, issuing certs — is written here. Nothing touches Vault unlogged.

Web UI & Connecting Phase 1 Services

Web UI: https://10.0.2.13:8200/ui/ — log in with the root token; expect a self-signed cert warning.

Authentik Secrets

```
vault kv put secret/authentik/admin
  username="admin@devsecops.com"
  password="..."
```

Database Credentials

```
vault kv put secret/database/mysql
  username="app_user"
  password="..." port=3306
```

Startup scripts fetch secrets with: curl --header "X-Vault-Token: $TOKEN" https://vault.devsecops.com/v1/secret/data/...

Validation Script & Troubleshooting

Validation checks (validate-chapter-10.sh):

- Service active
- Listening :8200
- Vault unsealed
- KV engine enabled
- Audit logging on

"Permission denied" on /var/lib/vault	`sudo systemctl status vault — confirm User=vault`
TLS certificate errors	`curl -k ... or export VAULT_SKIP_VERIFY=true`
Vault won't start	`sudo journalctl -xeu vault -n 50 — check perms, port, config syntax`

10.17 · CHAPTER SUMMARY

Secrets Are Now Centralized & Auditable

- Vault server — centralized secrets management
- TLS encryption — secrets encrypted in transit
- KV & PKI engines — generic storage + auto certs
- Access policies — fine-grained authorization
- Audit logging — every access recorded
- OIDC integration — apps authenticate via Authentik

NEXT: CHAPTER 11

Vault PKI

Certificate Management

- Generate TLS certs for services
- Automatic renewal — no manual expiration
- Certificate revocation lists (CRL)
- Compliance: audit every cert issuance